COREENGINE

AI: GOAP и Steering

Техническая документация архитектуры, ассетов и исполнения

Редакция от 8 сентября 2026 года • язык: русский

Репозиторий: `Fedorr47/CoreEngineModule` • ветка: `Deferred`

Проверенный commit: `55de4ab55c48af98c022534e2bc8013f11f7bd70` от 3 сентября 2026 года.

Документ описывает код этого снимка. Статус «реализовано» означает наличие соответствующего механизма в исходниках; результаты выполненных проверок перечислены отдельно. Примеры с условными числами отмечены как учебные. Схемы показывают ответственность и порядок вызовов, а не многопоточную модель.

Содержание

1. Назначение и границы подсистемы 3

2. GOAP, STRIPS и модель состояния 5

3. Цели, действия и три разных контракта 7

4. Поиск плана: стоимость и достижимость 9

5. Ассеты, registry и подготовленный template 11

6. Наблюдения, зависимости и подтверждённые события 14

7. Жизненный цикл решения и перепланирование 17

8. Исполнение, владение и порядок кадра 19

9. Два источника маршрута и общий MoveTo runtime 21

10. RouteFollower: углы, look-ahead и прибытие 23

11. Steering: направление и величина команды 25

12. FollowTarget и FleeTarget во времени 27

13. Obstacle avoidance: геометрия, память стороны и опора 28

14. Traversal, физика и фактически подключённые сервисы 32

15. Авторинг поведения и development-сценарии 34

16. Наблюдаемость и диагностика 37

17. Проверки и известные ограничения 39

18. Точки входа, термины и источники 41

1. Назначение и границы подсистемы

AI в CoreEngine связывает две задачи с разным масштабом времени. Первая — выбрать последовательность осмысленных действий: добраться до ресурса, собрать его, совершить покупку, пройти к цели. Вторая — превратить очередной шаг в движение персонажа и подтвердить результат в изменившемся мире. GOAP работает с фактами и последствиями действий. Steering вычисляет локальную команду движения. Между ними находятся исполнение плана, привязка семантического действия к конкретной задаче, поиск маршрута и обработка специальных переходов.

Это разделение позволяет менять содержание поведения, сохраняя общий runtime. Например, посещение маркера и восстановление пути после исчезновения цели используют один `move_to`. Access Key добавляет числовой ресурс, покупку и условия доступа. Maze проверяет другой вход в общую систему движения: маршрут поступает из NavMesh без GOAP-планирования.



Рисунок 1. Ответственность подсистем и обратная связь через наблюдение.

Факты в планировщике представляют знание агента, достаточное для выбора действий. Они не заменяют ECS, физическую сцену или уровень. Предсказанный эффект «маркер достигнут» становится частью поискового состояния, но сам по себе не перемещает персонажа и не подтверждает реальное прибытие. Подтверждение приходит из наблюдения. На этой границе строится всё дальнейшее описание.

Что изменилось в текущей архитектуре

В текущем снимке доменное поведение собирается из ассетов и зарегистрированных компонентов. Старые классы `GameplayAIAccessKeyDecision`, `GameplayAIAccessKeyObservation` и модуль `GameplayAIDecisionComposition` удалены. Их имена полезны при чтении истории, но не подходят как точки расширения действующей версии.

Изменение	Практический смысл
Каталог решений и подготовленный template	Определение GOAP компилируется при регистрации; состояние агента создаётся отдельно
Типизированные параметры компонентов	Новый тип регистрирует parser и compiler; общая композиция не перебирает закрытый список доменов
Зависимости bool- и int-наблюдений	Порядок вычисления фактов определяется их производителями и потребителями
`continuationConditions`	Действующий шаг может потерять право продолжаться, даже если успешно стартовал
Обобщённый `purchase` и `receiptId`	Покупка больше не зашита в событие Access Key; несколько результатов различаются идентификатором квитанции

Документ предназначен для разработчика Gameplay/AI и автора development-сценариев. Разделы 2–7 объясняют решение и факты, 8–14 — исполнение и движение, 15–18 — авторинг, диагностику и проверку. Для первого знакомства полезно пройти их по порядку: ошибки движения часто начинаются с неверного контракта наблюдения.

Код: GameplayAIBuiltinDecisions.cppm, GameplayGOAPAssetComposition.cppm, GameplayAIDecisionCreationContext.cppm.

2. GOAP, STRIPS и модель состояния

GOAP и STRIPS — связь в одном абзаце

GOAP, Goal-Oriented Action Planning, строит последовательность действий от текущего состояния к выбранной цели; STRIPS даёт классическую основу для описания таких действий через предусловия и изменения фактов. В CoreEngine это проявляется в явных условиях применимости, эффектах и проверке достижения цели, но реализация расширяет простую булеву модель целочисленными ресурсами, стоимостями и контрактами исполнения. Поэтому точнее говорить о GOAP-планировщике с моделью действий в духе STRIPS: он ищет подходящую цепочку в символическом состоянии, а runtime отдельно выполняет её в игровом мире и проверяет, сохраняется ли основание продолжать. Историческая основа: Fikes и Nilsson, STRIPS.

Состояние с фиксированной ёмкостью

`AIAgentWorldState` хранит до 128 булевых фактов и до 128 целочисленных фактов `int32`. Это две независимые области идентификаторов: булев факт с индексом 0 и целочисленный факт с индексом 0 обозначают разные ячейки. Сильные типы идентификаторов защищают границу между ними на уровне API. Булевы значения находятся в bitset, числа — в массиве фиксированного размера.

$$S = (B, I), \qquad B \in \{0,1\}^{128}, \qquad I \in \mathbb{Z}_{32}^{128}$$

Здесь S — полное поисковое состояние, B — булевы значения, I — знаковые 32-битные числа. Формула описывает ёмкость контейнера, а не требование объявить 256 фактов. Автор объявляет только нужные факты. Metadata определения связывает их имена с типизированными ID.

В текущем контейнере нет третьего значения «неизвестно»: `false` и 0 являются значениями, а не признаками отсутствующего наблюдения. Если домен различает «цель отсутствует» и «цель ещё не проверена», ему нужны отдельные объявленные факты и наблюдение, которое поддерживает их смысл. Не стоит использовать неинициализированную ячейку как скрытое состояние знания.

Компилятор назначает ID фактов по порядку объявления внутри соответствующего типа, ID целей — по порядку целей, ID контекстов — по порядку контекстов. Эти значения удобны внутри скомпилированного определения, но не являются устойчивыми внешними ключами для сохранений между редакциями ассета. В авторинге используются имена; семантические ID действий предоставляет registry.

Пример: ресурс и доступ

В Access Key целочисленный факт `coins` описывает ресурс. Булевы факты отражают подтверждённый сбор отдельных предметов, получение доступа и состояние местоположения. Такое представление даёт планировщику возможность сравнивать планы с покупкой при цене 2, не создавая отдельный булев факт для каждого возможного количества монет.

Целочисленное пространство всё же требует дисциплины. Действие с неограниченным увеличением счётчика порождает очень много различных состояний. Даже фиксированный `int32` не превращает поиск в практически ограниченную задачу. В модели

должны быть конечные источники ресурса, условия применимости или иные ограничения, которые отражают реальную игру.

Равенство и хеширование учитывают полное состояние; используется FNV-1a. Планировщик различает состояния по значениям фактов, а не по тому, каким маршрутом они были получены. Поэтому два плана, пришедшие к одинаковым B и I, конкурируют по стоимости. Данные, от которых зависит будущая применимость действий, должны быть представлены в состоянии либо явно проверяться на границе исполнения.

Код: AIAgentWorldState.cppm, GameplayGOAPDefinitionCompiler.cppm, access_key.goap.json.

3. Цели, действия и три разных контракта

Цель описывает требуемые булевы значения. Выбор цели использует базовую оценку и правила, зависящие от булевых фактов; уже достигнутые цели пропускаются. Выбор цели и поиск плана — отдельные операции. В текущем runtime не следует ожидать автоматического перебора всех менее приоритетных целей, если план для выбранной цели не найден.

У действия есть семантический тип, контекст, стоимость, предусловия и эффекты. Например, `move_to` — тип возможности движения, а `visitMarker` или `destination` — конкретный контекст. Один provider может обслуживать несколько контекстов одного типа. Для покупки семантическое имя теперь `purchase`; `buy_key` остаётся именем контекста в Access Key.

Часть действия	Когда нужна	Что означает
`preconditions` и числовые предусловия	Поиск и проверка ещё не начатого шага	Можно ли применить действие к этому состоянию
`effects` и числовые эффекты	Построение предсказанного состояния	Что модель ожидает после успеха
`continuationConditions` и числовые условия продолжения	Проверка уже выполняемого шага	Сохранилось ли основание продолжать действие

Для булевых условий проверяется равенство заданному значению. Числовые условия задают сравнение с константой; числовые эффекты изменяют числовой факт через предусмотренную операцию. Реализация проверяет диапазон целочисленного результата: переполнение не должно превращаться в допустимую ветвь поиска.

$$\text{applicable}(a, S) = \bigwedge_{p \in P_a} p(S), \qquad S' = T_a(S)$$

P_a — совокупность стартовых условий действия a; T_a — применение его предсказанных эффектов к копии S. Полученное S′ используется только в поиске. Например, модель покупки проверяет `coins >= 2`, затем предсказывает уменьшение `coins` на 2 и получение доступа. В реальном исполнении эти изменения подтверждаются через событие покупки и ledger.

Зачем выделены условия продолжения

Стартовое условие может быть расходуемым. Действие способно занять ресурс или изменить собственный контекст после начала. Повторная проверка всех стартовых условий каждый кадр в таком случае прерывала бы корректное исполнение. Условия продолжения задают более узкий контракт: какие из стартовых оснований обязаны оставаться истинными весь шаг.

В текущей реализации каждое условие продолжения обязано в точности присутствовать среди предусловий. Для числа совпадают факт, операция сравнения и значение. Это не механизм произвольной дополнительной логики после старта. В поиске продолжение не образует отдельный фильтр будущих состояний: при входе в действие соответствующие условия уже покрыты предусловиями.

В Target Recovery у `move_to` и предусловие, и условие продолжения содержат `goalAvailable == true`. Если Interaction Point цели исчезает во время движения, факт меняется, выполняемый шаг становится недопустимым, а runtime отменяет его, когда заменить план нечем. После возвращения цели тот же живой decision instance способен построить новый план.

Код: AIDecisionContracts.cppm, AIStateOperations.cppm, AIGoalSelection.cppm, target_recovery.goap.json.

4. Поиск плана: стоимость и достижимость

`FindAIPlan` выполняет прямой поиск по состояниям с минимальной накопленной стоимостью. В этом снимке это uniform-cost search, то есть схема Дейкстры; эвристики A* нет. Начальный узел содержит наблюдаемое состояние и пустой план. Из узла применяются подходящие действия, их эффекты создают состояния-потомки, а стоимость увеличивается на стоимость шага.

$$g(S') = g(S) + c(a), \qquad c(a) \geq 0$$

g — уже накопленная стоимость пути в пространстве состояний; c(a) — конечная неотрицательная стоимость действия. Она не обязана измеряться в метрах или секундах. Важно, чтобы сравниваемые действия использовали согласованный смысл стоимости. Иначе оптимальный по числу результат окажется не тем поведением, которое ожидал автор.

Очередь упорядочена по стоимости, затем по стабильному порядку вставки. Действия рассматриваются в порядке пары `(actionId, contextId)`. Для каждого полного состояния хранится лучшая найденная стоимость. При улучшении в очередь добавляется новая запись; старую не удаляют через decrease-key. После извлечения запись пропускается, если её стоимость уже не совпадает с лучшей.

```
push(startState, cost = 0, steps = [])
while queue is not empty:
    node = popLowestCost()
    if node.cost != bestCost[node.state]: continue
    if goalSatisfied(node.state): return node.steps
    for action in orderedActions:
        if not applicable(action, node.state): continue
        next = applyToCopy(node.state, action.effects)
        if next is invalid: continue
        relax(next, node.cost + action.cost,
              node.steps + [action])
return noPlan
```

Псевдокод передаёт цикл поиска, а не является копируемым API. Реальная реализация дополнительно проверяет входные данные, конечность стоимости, диапазоны и контракт условий продолжения.

Небольшой пример стоимости

Предположим, учебная модель предлагает два способа получить доступ. Первый путь стоит 3 за перемещение к ресурсу, 2 за покупку и 8 за путь к выходу: всего 13. Второй путь даёт другую цепочку стоимостью 18. Поиск выбирает 13, если обе цепочки приводят к одной цели и все условия выполнены. Это условные числа, не извлечённые из карты Access Key.

При `routeCost: true` конкретный MoveTo provider может заменить стоимость контекста стоимостью найденного пути по авторскому графу. Это происходит в копии определения для экземпляра. Цена маршрута не пересчитывается непрерывно вслед за физическим перемещением агента и не является прогнозом времени на обход динамических препятствий.

Если начальное состояние уже удовлетворяет цели, результат — пустой план. Недопустимый вход либо недостижимая цель дают отсутствие плана. Не следует

смешивать этот результат с явной отменой поведения: runtime может повторить поиск на следующем обновлении после изменения наблюдения.

Стоимость самого поиска

Узел хранит копию полного состояния и полного вектора шагов. Поэтому цена поиска определяется не только количеством состояний, но и длиной копируемых планов. В коде нет отдельного лимита раскрытий или временного бюджета. Гарантированно малый домен важнее формальной конечности контейнера. При неуспешном поиске retained instance может пытаться планировать снова каждый update; backoff также не реализован.

Практическая работа над производительностью начинается с измерения числа раскрытых состояний и частоты перепланирования, затем — с сокращения лишних фактов и контекстов. Эвристика, хранение parent-ссылок и бюджетированный поиск являются возможными направлениями развития, а не свойствами текущей реализации.

Код: AIPlanner.cppm, AIStateOperations.cppm, GameplayGOAPDecisionTemplate.cppm.

5. Ассеты, registry и подготовленный template

Сборка поведения начинается в каталоге `assets/ai/decisions/catalog.json`. Запись связывает ID решения с behavior и level bindings. Behavior указывает GOAP definition, наблюдения, capabilities, реакции и при необходимости граф маршрутов. Это несколько файлов с разной ответственностью, а не одно взаимозаменяемое описание AI.

Вид файла	Ответственность
`.goap.json`	Факты, цели, контексты действий, условия, эффекты и стоимость
`.behavior.json`	Компоненты наблюдения, исполнения и реакции; ссылки на definition и routes
`.bindings.json`	Семантическая роль → именованный node уровня; ссылки на traversal
`.routes.json`	Именованные узлы и направленные рёбра авторского графа
`.scenario.json`	Development lifecycle: setup, start, update, stop, reset
`.level.json`	Геометрия и сущности мира, параметры персонажа и прочие данные уровня

Регистрация компонента

`GameplayGOAPCompositionRegistry` регистрирует observation, capability или reaction вместе с parser и compiler. Parser получает JSON на стадии загрузки и возвращает владеющий типизированный payload. Общий контейнер параметров использует `std::any`; compiler извлекает ожидаемый тип через `Parameters<T>`. Это позволяет добавить расширение без правки закрытого variant со всеми встроенными типами.

`std::any` здесь относится к авторингу. Рабочее состояние GOAP по-прежнему состоит из фиксированных bool/int фактов. Новый JSON-набор для уже зарегистрированного типа может быть достаточен для нового поведения; совершенно новый тип наблюдения или исполнения требует C++-реализации, разбора параметров и регистрации. Одно только незнакомое поле `type` не создаёт механизм автоматически.

Registry отвергает конфликтующую регистрацию. Для capability также нужен уникальный семантический ID действия. Поля встроенных компонентов проверяются их parser, поэтому опечатки не стоит считать неявными настройками по умолчанию.

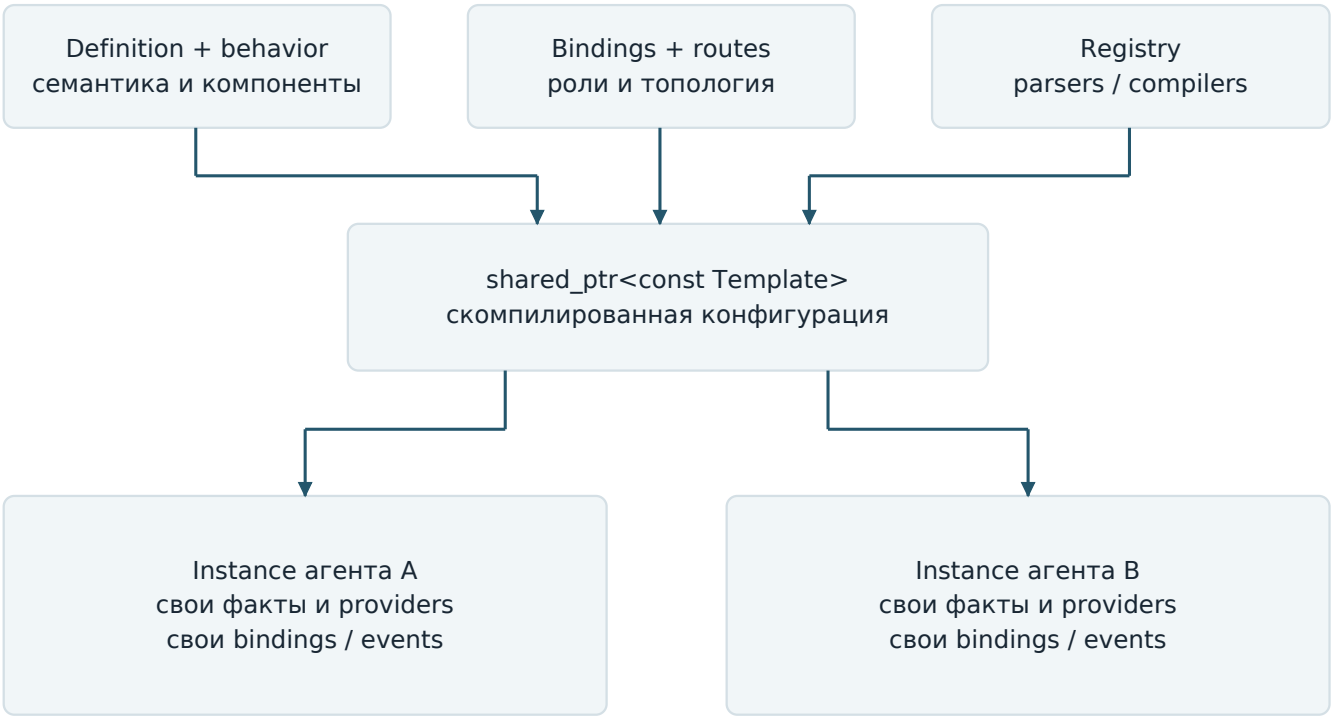


Рисунок 2. Общий template и принадлежащие отдельным агентам данные.

Две стадии подготовки

`GameplayGOAPDecisionTemplate` не содержит живой world или конкретного агента. Он хранит снимок конфигурации, registry и скомпилированное определение. На этой стадии проверяются модель `goap`, известность типов компонентов, наличие нужного route graph, уникальность capability-контекстов и покрытие каждого контекста GOAP подходящим исполнителем.

`RegisterGameplayAIDecisionAssets` подготавливает все записи во временной копии factory registry. Только после успешной загрузки и компиляции публикуется новое содержимое registry. Такая транзакционная регистрация не оставляет половину каталога при ошибке в последнем behavior. Factory захватывает `shared_ptr<const ...>` template; созданные ранее instances продолжают владеть своей конфигурацией. Автоматического hot reload живых решений этот механизм не даёт.

При создании instance bindings разрешают роли относительно уровня. Имя node должно находить однозначную живую цель, позиция должна быть конечной, связь node с entity не должна быть неоднозначной. Роль может представлять позицию без entity, если конкретный компонент не требует сущности. Затем создаются observers, reactions и providers, строится порядок наблюдений и собираются action bindings.

Provider может вернуть overrides стоимости для пары `(action, context)`. `InstanceDefinition` копирует подготовленное определение и применяет только допустимые, конечные и неотрицательные overrides. Неизвестная или повторённая пара отвергается. Общий template при этом не меняется.

Существенная граница: роль — это снимок привязки

Текущая композиция кеширует позицию из `LevelAsset.nodes[node].transform.position` и найденный entity handle. Встроенные spatial observations и MoveTo используют эту сохранённую позицию. Они не являются универсальным трекером движущейся цели по её текущему transform. Удаление сущности и создание новой с другим handle тоже не приводит к автоматическому перепривязыванию роли.

Именно поэтому сценарий восстановления временно меняет доступность прежней цели, а не демонстрирует произвольное обнаружение нового объекта. Для движущейся цели есть отдельный FollowTarget runtime; его смысл не стоит переносить на `move_to` из ассетов.

Код: GameplayGOAPCompositionRegistry.cppm, GameplayAIDecisionAsset.cppm, GameplayGOAPDecisionTemplate.cppm, GameplayGOAPAssetComposition.cppm.

6. Наблюдения, зависимости и подтверждённые события

Наблюдение переводит состояние мира и события в факты агента. В композиции у каждого объявленного bool- и int-факта должен быть ровно один writer. Неизвестный output, дублирование writer или отсутствие producer считаются ошибкой подготовки. Метаданные inputs задают зависимости: потребитель должен запускаться после производителя нужного факта.

`OrderGameplayGOAPObservations` строит топологический порядок. На каждой итерации выбирается первый готовый observer из исходного порядка. Это сохраняет предсказуемый порядок между независимыми готовыми компонентами. Если следующего готового observer нет, оставшаяся зависимость циклична либо неразрешима, и композиция отвергается.

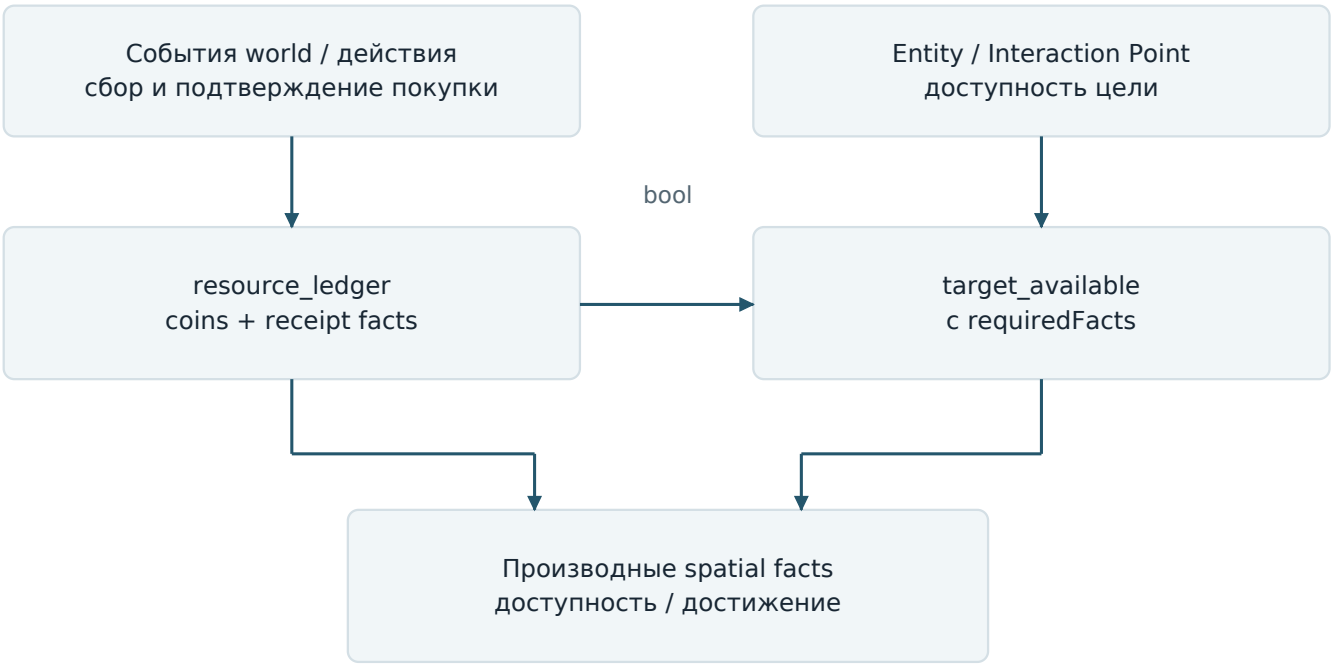


Схема зависимости; конкретные inputs задаёт компонент

Рисунок 3. Зависимости наблюдений определяются метаданными, а не текущими значениями фактов.

Наличие producer означает, что факт вычислен, даже если его значение `false` или 0. Зависимость по integer работает так же, как по boolean. Она не превращает нулевой баланс в «не готово». При добавлении нового компонента важно объявить все факты, которые он читает, и все, за которые отвечает.

Встроенные наблюдения

Тип	Источник факта	Важная особенность
`target_available`	Живость привязанной entity и опциональный Interaction Point	Может учитывать `requiredFacts` и latch; не выполняет visibility raycast
`within_distance`	Квадрат 3D-расстояния агента до сохранённой позиции роли	Положительный конечный radius; опциональный latch
`nearest_location`	Ближайшая из сохранённых позиций в пределах radius	One-hot результат; вне радиуса сохраняется последнее подтверждённое место
`pickup_available`	Живой ещё не собранный pickup	При учёте reservations собственная бронь допустима, чужая блокирует
`resource_ledger`	События сбора и покупки	Обновляет числовой баланс и факты подтверждённых операций

Latch превращает факт в память о событии: однажды подтверждённое значение может оставаться истинным. Это полезно для «побывал в точке», но имеет другой смысл, чем «находится в точке сейчас». У `nearest_location` также есть память последнего подтверждённого места; при равной дистанции из-за сравнения <= выбирается более поздний элемент списка. Не следует полагаться на порядок таких элементов как на содержательное правило поведения.

Ledger и покупка

В Access Key три pickup могут выдать по одной монете, а receipt `unlock` стоит 2. Ledger принимает `PickupCollected` только для своего агента и настроенного предмета; уже подтверждённый сбор не начисляется повторно. Изменение считается через более широкий целочисленный промежуточный результат. Недопустимый баланс отвергается до частичного подтверждения факта сбора.

`PurchaseRuntime::Start` проверяет тип действия, живого продавца, ещё не полученный результат, достаточный баланс и близость агента к сохранённой позиции продавца. При успехе он создаёт `ResourcePurchased` и завершает действие. Сам runtime покупки не переписывает наблюдаемые факты. Ledger сопоставляет instigator, subject и `receiptId`, затем списывает ресурс и подтверждает результат.

`receiptId` нужен, потому что один vendor может продавать несколько разных результатов. Идентификаторы receipts уникальны внутри behavior. Проверка capability также связывает модель покупки с исполнением: ожидается один нужный bool-эффект, числовое списание ровно цены и числовое предусловие достаточного баланса. Иначе планировщик мог бы «покупать» по одной модели, а мир — по другой.

Отдельная реакция `hide_on_purchase` получает подтверждённую покупку и формирует `HideEntityRequested`. Скрываемая сущность может отличаться от vendor. Реакция срабатывает один раз на instance. Визуальное последствие таким образом отделено от учёта ресурса, но связано с тем же подтверждённым результатом.

Два прохода событий

Instance сначала наблюдает внешние события, затем запускает реакции, обновляет GOAP, наблюдает события созданных действий и вызывает реакции на них. Это позволяет покупке, завершившейся внутри update, обновить ledger в том же проходе.

Однако `ObserveActionEvents` — отдельный виртуальный метод; его реализация по умолчанию пуста. Он не вызывает `Observe` автоматически. Spatial-компоненты и ledger поддерживают этот проход, а `pickup_available` не переопределяет его. При написании observer нужно явно решить, какие результаты должны быть видны после событий действия и какие будут пересчитаны в начале следующего update.

Код: GameplayGOAPObservationOrder.cppm, GameplayGOAPSpatialComponents.cppm, GameplayGOAPResourceComponents.cppm, GameplayGOAPDecisionInstance.cppm, GameplayWorldEvent.cppm.

7. Жизненный цикл решения и перепланирование

`AIDecisionRuntime` координирует выбор цели, текущий план и bridge к `AISystem`. Он не заменяет action runtime. В начале update синхронизируется статус уже исполняемого шага; затем выбирается цель. Если подходящей неудовлетворённой цели нет, активное исполнение отменяется, план очищается, а статус решения становится `Succeeded`.

Для неизменившейся цели runtime различает две проверки. Перед стартом следующего шага он проверяет оставшийся план целиком: последовательно проверяет предусловия, применяет предсказанные эффекты и убеждается, что итог удовлетворяет цели. Для уже выполняемого шага проверяются только его continuation conditions. Сохранение применимости не означает повторного поиска более дешёвого плана.

Если нужен новый план, поиск выполняется до отмены старого исполнения. При найденной замене старое исполнение отменяется, новый план устанавливается и переходит в состояние готовности. Конкретный шаг запускается последующим update. Такое разделение нужно учитывать в диагностике: только что построенный план может ещё не иметь запущенной задачи в `AISystem`.



Рисунок 4. Восстановление уже существующего решения при временной недоступности цели.

Неуспех, отмена и сохранение старого исполнения

Ситуация	Поведение текущего runtime
Цель прежняя, план применим	Существующее исполнение сохраняется
Текущий шаг потерял continuation, замены нет	Активное исполнение отменяется; решение получает `Failed`
Новая цель, замены нет, есть допустимое для этой ветви активное исполнение	Старое исполнение может быть сохранено
Вызван явный `Cancel`	Устанавливается признак explicit cancellation; автоматический update больше не возобновляет решение
`GameplayRuntime::CancelAIDecision`	Instance отменяется и удаляется из активной коллекции

В третьей строке есть тонкость текущего кода: проверка invalidation выполняется только при неизменившейся цели. На ветви смены цели runtime не проводит заново continuation-проверку старого шага перед его возможным сохранением. Поэтому утверждение «при неуспешной смене цели старый план всегда полностью перепроверен» было бы неверным.

Явная отмена и `Failed` имеют разный смысл. Retained instance со статусом `Failed` продолжает получать updates и может восстановиться. Explicit cancellation блокирует такое автоматическое продолжение; успешный явный вызов `Plan(...)` снимает этот флаг на уровне decision runtime. После удаления через GameplayRuntime для нового старта создаётся другой instance.

Target Recovery по шагам

Behavior `target_recovery` содержит `target_available`, проверяющий Interaction Point, и `within_distance` с радиусом 0.4. Действие идёт из `origin` в `destination`, использует slowing radius 1.5 и требует `goalAvailable` как при старте, так и при продолжении.

Development-сценарий временно убирает доступность цели примерно на половине пути на 5 секунд. Это та же сущность с изменённым состоянием доступности. При следующем наблюдении `goalAvailable` становится ложным. Продолжение теряет основание, альтернативного плана нет, движение отменяется. Instance остаётся в GameplayRuntime; после возвращения Interaction Point наблюдение снова разрешает движение, план строится и запускается повторно.

Это восстановление работает для уже зарегистрированного активного instance. `StartAIDecision` выполняет первый update синхронно и не сохраняет instance, если старт заканчивается неуспехом, отменой или отсутствием старта. Следовательно, запуск с изначально недоступной целью возвращает неуспех и сам по себе не создаёт ожидающий instance, который проснётся через пять секунд.

Активные решения обновляются в стабильном порядке по entity handle. Terminal-статус сам по себе не удаляет запись. Это обеспечивает повторное наблюдение и восстановление, но означает повторные попытки поиска без backoff. Стабильный порядок может влиять на победителя при конкуренции за ресурс; он не является механизмом справедливого распределения.

Код: AIDecisionRuntime.cppm, GameplayRuntime_PublicApi.inl, GameplayRuntime_FrameLifecycle.inl, target_recovery.behavior.json, ai_goap_target_recovery.scenario.json.

8. Исполнение, владение и порядок кадра

План состоит из семантических шагов. `AIPlanExecution` хранит ход исполнения, bridge сопоставляет шаг с binding, а binding создаёт конкретный action runtime для `AISystem`. Именно этот runtime получает `Start`, `Tick` и `Cancel`, выдаёт промежуточный или окончательный результат и меняет команду агента либо создаёт событие.

Такой контракт поддерживает и длительное движение, и мгновенную покупку. Успешный `Start` покупки может сразу вернуть `Succeeded`. Начало движения обычно возвращает `Running`, после чего результат достигается серией ticks. Статус действия сообщает о работе исполнителя; истинность цели по-прежнему определяется наблюдаемыми фактами.

Время жизни объектов

`GameplayGOAPDecisionInstance` владеет контекстом, reactions, буферами событий и GOAP-обёрткой. Providers принадлежат composed context; bindings заимствуют ссылки на них. Порядок полей обеспечивает уничтожение GOAP и bindings до контекста и событий, на которые они ссылаются. Копирование и перемещение instance запрещены.

Общий template разделяется как неизменяемый объект, но факты, состояние providers, latch реакций и буферы событий принадлежат конкретному агенту. Это предотвращает случайное разделение прогресса между двумя решениями. Разделяемый `const` template не делает mutable runtime потокобезопасным: ссылки на world и сервисы остаются частью контракта вызывающего кода.

Владелец решения должен отменить активные задачи `AISystem` до уничтожения instance и его providers. Публичные мутации GameplayRuntime проверяют принадлежность своему потоку. Текущий путь обновления не следует описывать как параллельный AI job graph: наблюдение и запуск действий идут в явном последовательном порядке.

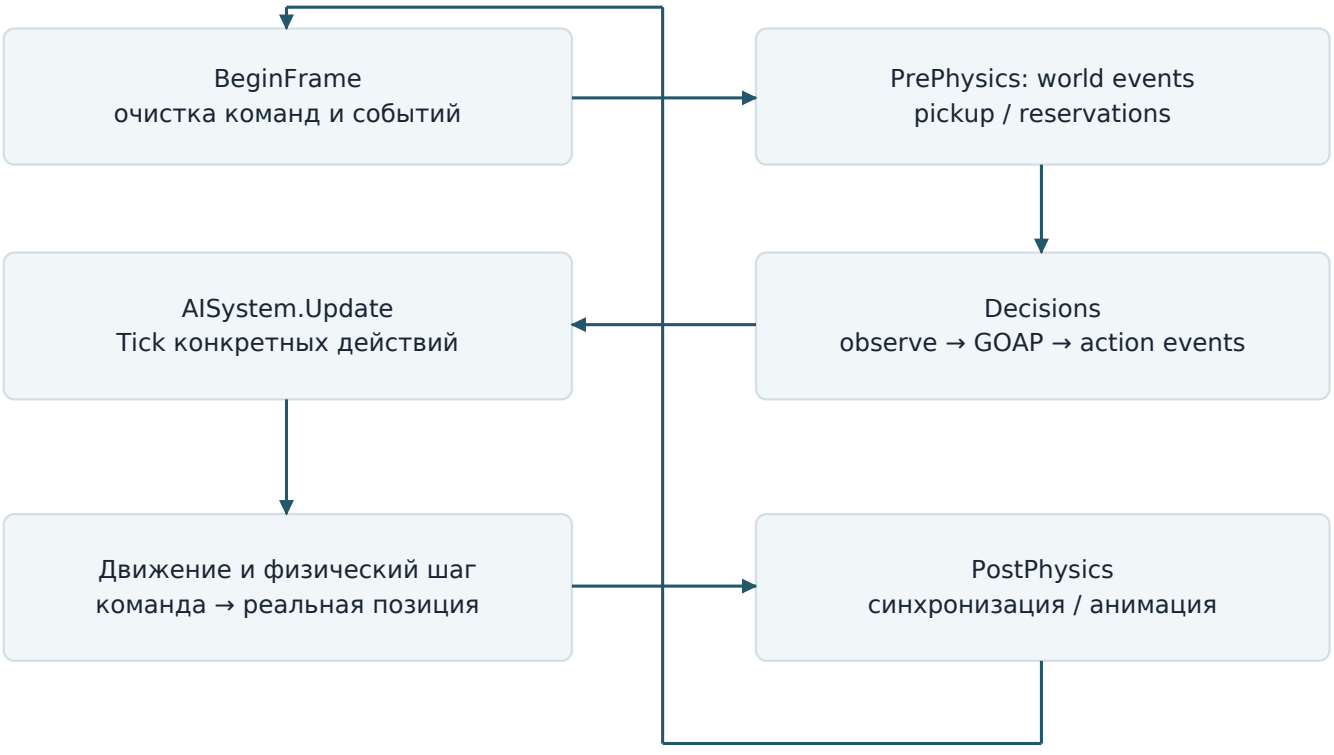


Рисунок 5. Порядок игрового кадра и место обратной связи от физики.

Что происходит за кадр

`BeginFrame` очищает события кадра и временные команды персонажей. В `PrePhysicsUpdate` после подготовки world и пользовательского ввода очищаются недействительные reservations, обновляется pickup system, затем решения AI. События сбора уже доступны наблюдению; скрытие собранных предметов также обрабатывается в этом участке.

После решений выполняется `aiSystem.Update(delta)`: конкретные задачи движения выдают команду текущего кадра. Затем идут дальнейшие gameplay-операции и обновление движения персонажа. Физический шаг выполняется в App после pre-physics; post-physics синхронизирует состояние движения и transforms и передаёт gameplay-состояние анимации. Следующее наблюдение получает уже обновлённый мир.

Внутри одного decision update порядок ещё точнее: очистка reaction events → `Observe` внешних событий → reactions внешних событий → очистка runtime events → GOAP update → `ObserveActionEvents` → reactions событий действия → добавление выходных событий. Входной и выходной массивы могут совпадать, поэтому запись в общий output откладывается до окончания чтения input. Это локальное правило корректности буферов, а не гарантия межпоточной синхронизации.

Почему команда повторяется каждый tick

Follow/Flee могут пересчитывать steering с отдельной периодичностью, но `GameplayCharacterCommandComponent` живёт один кадр и очищается в `BeginFrame`. Следовательно, сохранённый steering sample нужно применять снова на каждом tick. Если записывать команду только при новом вычислении, агент будет получать движение лишь в отдельных кадрах. Текущие runtimes переиздают кешированное намерение и обновляют desired facing для ненулевого движения, включая ticks между пересчётами.

Код: AIPlanExecution.cppm, AIPlanExecutionBridge.cppm, AISystem.cppm, GameplayGOAPDecisionInstance.cppm, GameplayRuntime_FrameLifecycle.inl, AppLifecycle_TickImpl.inl.

9. Два источника маршрута и общий MoveTo runtime

Маршрут отвечает на пространственный вопрос: через какие точки и переходы нужно пройти. Он отличается от GOAP-плана, который отвечает на вопрос о последовательности действий. Один шаг `move_to` может содержать длинный маршрут; весь GOAP-план может включать несколько разных маршрутов и покупку между ними.

В CoreEngine общий `GameplayRoute` содержит точки и аннотации сегментов. Для N точек требуется N−1 аннотаций. Аннотация может указывать traversal link. Follower владеет маршрутом по значению; первая точка — его стартовый anchor. Он не добавляет автоматически сегмент от текущей позиции персонажа к этому anchor.

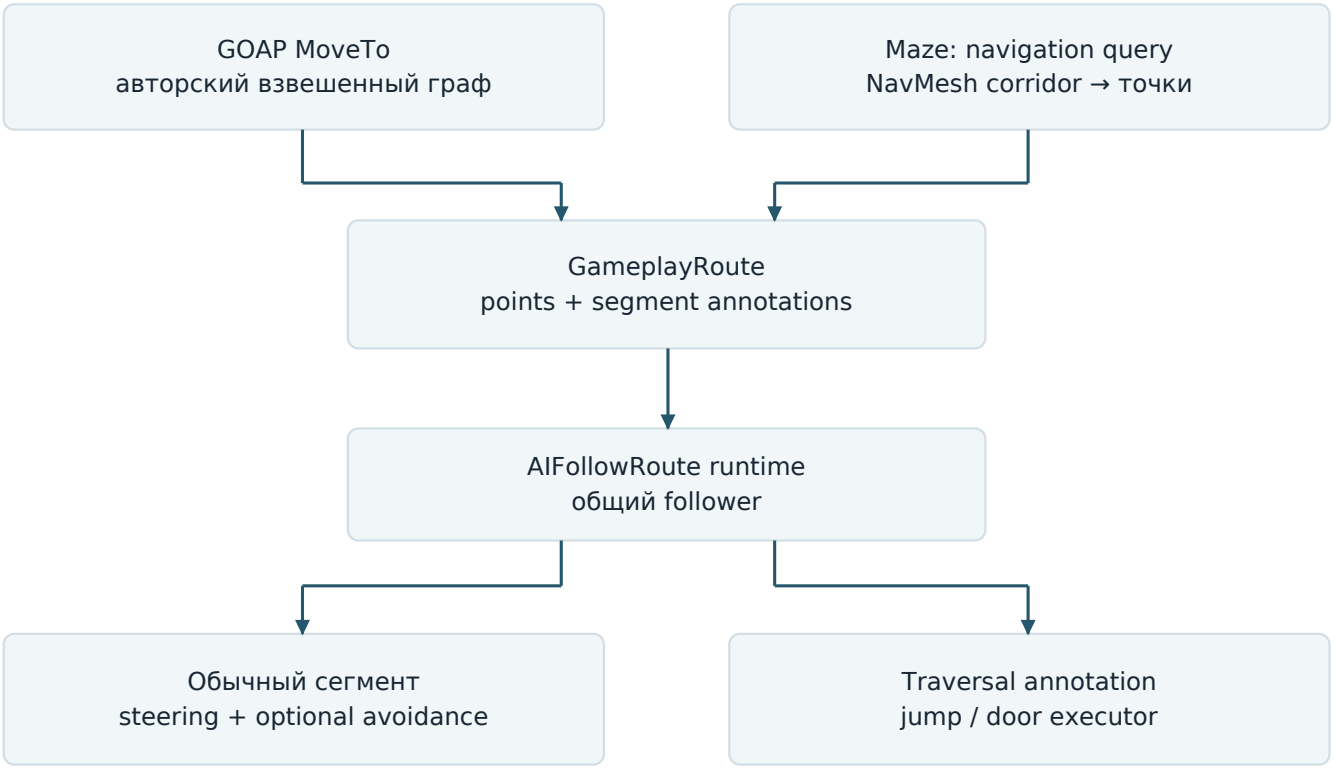


Рисунок 6. Авторский граф и NavMesh сходятся в общем исполнителе маршрута.

Авторский граф в GOAP-композиции

Файл `.routes.json` задаёт именованные узлы и направленные рёбра. При создании instance узлы связываются с сохранёнными позициями ролей. Если стоимость ребра не задана, используется евклидово расстояние; если задана — берётся авторская стоимость. Направленность существенна: ребро A → B не обещает возможность вернуться B → A.

MoveTo provider разрешает конкретные source и target и проверяет, что нужный маршрут существует. `routeCost: true` переносит стоимость найденного пути в GOAP-контекст. Это связывает два поиска по данным, но не объединяет их пространства состояний: сначала подготовлена пространственная цена, затем GOAP сравнивает действия по этой цене.

Если behavior не содержит route graph, для контекста создаётся граф из двух узлов с одним ребром стоимостью 1. Это прямой маршрут между привязанными позициями. В таком режиме `routeCost: true` запрещён; стоимость GOAP остаётся определённой в definition. Прямой fallback не обращается к NavMesh и не гарантирует путь вокруг стен.

Параметр `sourceRadius` ограничивает место допустимого старта. При положительном значении фактическая позиция должна быть достаточно близка к configured start, и ближайший узел графа должен совпадать с ним. При 0 эта проверка пропускается. Это позволяет Target Recovery заново начать движение из промежуточной позиции, хотя source role осталась прежней.

Reservation цели

При `reserveTarget` нужен живой Interaction Point. Обёртка binding пытается зарезервировать его на старте, освобождает reservation при terminal-результате или отмене и завершает шаг неуспехом при потере владения. Наблюдение `pickup_available` может учитывать те же reservations: чужая бронь делает предмет недоступным, собственная остаётся совместимой с продолжением.

Планировщик не бронирует все будущие шаги заранее. Между прогнозом и реальным стартом другой агент способен изменить доступность. Reservation принадлежит контракту исполнения, а новое наблюдение позволяет принять следующее решение после конфликта.

NavMesh и Maze

Maze запускает `StartAIFollowRoute` напрямую после навигационного запроса. App собирает геометрию статических meshes в мировых координатах; навигационный профиль использует физические параметры персонажа. Запрос находит стартовый и целевой полигоны, строит Detour corridor и извлекает straight path, который преобразуется в `GameplayRoute`.

Завершённость corridor проверяется: `NoPath` и `BufferTooSmall` не следует интерпретировать как успешно найденный полный путь к цели. Само наличие нескольких точек ещё не доказывает достижимость финиша. Семантика библиотечных запросов описана в Detour: dtNavMeshQuery; преобразование результата и политика принятия пути определены кодом CoreEngine.

В Maze настроены `searchExtents = (1, 2, 1)`, acceptance radius 0.25, slowing radius 0.9 и ходьба. Физический radius агента — 0.35, высота цилиндра — 1.1, полная высота — 1.8, допустимый склон — 45°, шаг — 0.35, масса — 80. Эти значения относятся к текущему сценарию и уровню, а не являются общими константами AI.

Код: GameplayGOAPMoveToComponent.cppm, AIMoveToActionBinding.cppm, GameplayRoute.cppm, GameplayRouteSearch.cppm, LevelNavigation.cppm, Navigation.cpp, navigation_pathfinding_maze.scenario.json, navigation_pathfinding_maze.level.json.

10. RouteFollower: углы, look-ahead и прибытие

`GameplayRouteFollower` хранит индекс текущего сегмента и выбирает локальную цель. Для промежуточного waypoint slowing radius приравнивается к acceptance radius: персонаж не должен полностью тормозить перед каждой точкой коридора. Для последнего сегмента используется настроенный slowing radius, чтобы замедлиться перед финишем.

Когда промежуточный угол достаточно близок, look-ahead переносит steering target на следующий сегмент. При значении 0 сохраняется движение непосредственно к waypoint; значение по умолчанию — 0.75. Look-ahead не перескакивает на отсутствующий следующий сегмент или через следующий traversal-сегмент.

$$q = c + \hat{u}_{out}\min(L - \|p - c\|, \ell_{out})$$

Формула применяется при расстоянии до угла меньше L и допустимых невырожденных сегментах. p — позиция агента в XZ, c — угол, L — look-ahead distance, □out — длина следующего сегмента, ûout — его единичное направление. Чем ближе агент к углу, тем дальше виртуальная точка q продвигается по выходящему отрезку. Например, при L = 0.75 и расстоянии 0.25 цель находится на 0.5 дальше угла, если следующий отрезок достаточно длинный.

Когда считается пройденным угол

Одного попадания в маленький круг acceptance недостаточно для плавного срезания угла. Текущая реализация также допускает смену сегмента после пересечения плоскости перехода внутри локального envelope. Нормаль этой плоскости строится из суммы единичных направлений входящего и исходящего отрезков.

$$\hat{n} = \frac{\hat{u}_{in} + \hat{u}_{out}}{\|\hat{u}_{in} + \hat{u}_{out}\|}, \qquad (p - c) \cdot \hat{n} \geq 0$$

Для поворота с +X на +Z нормаль пропорциональна (1, 1) в плоскости XZ, а граница имеет вид x + z = 0 относительно угла. Точка (−0.2, 0.3) уже пересекла эту границу, хотя ещё не пересекла старую плоскость x = 0. В пределах look-ahead envelope это позволяет продолжать движение по выходящему сегменту без возврата к waypoint.

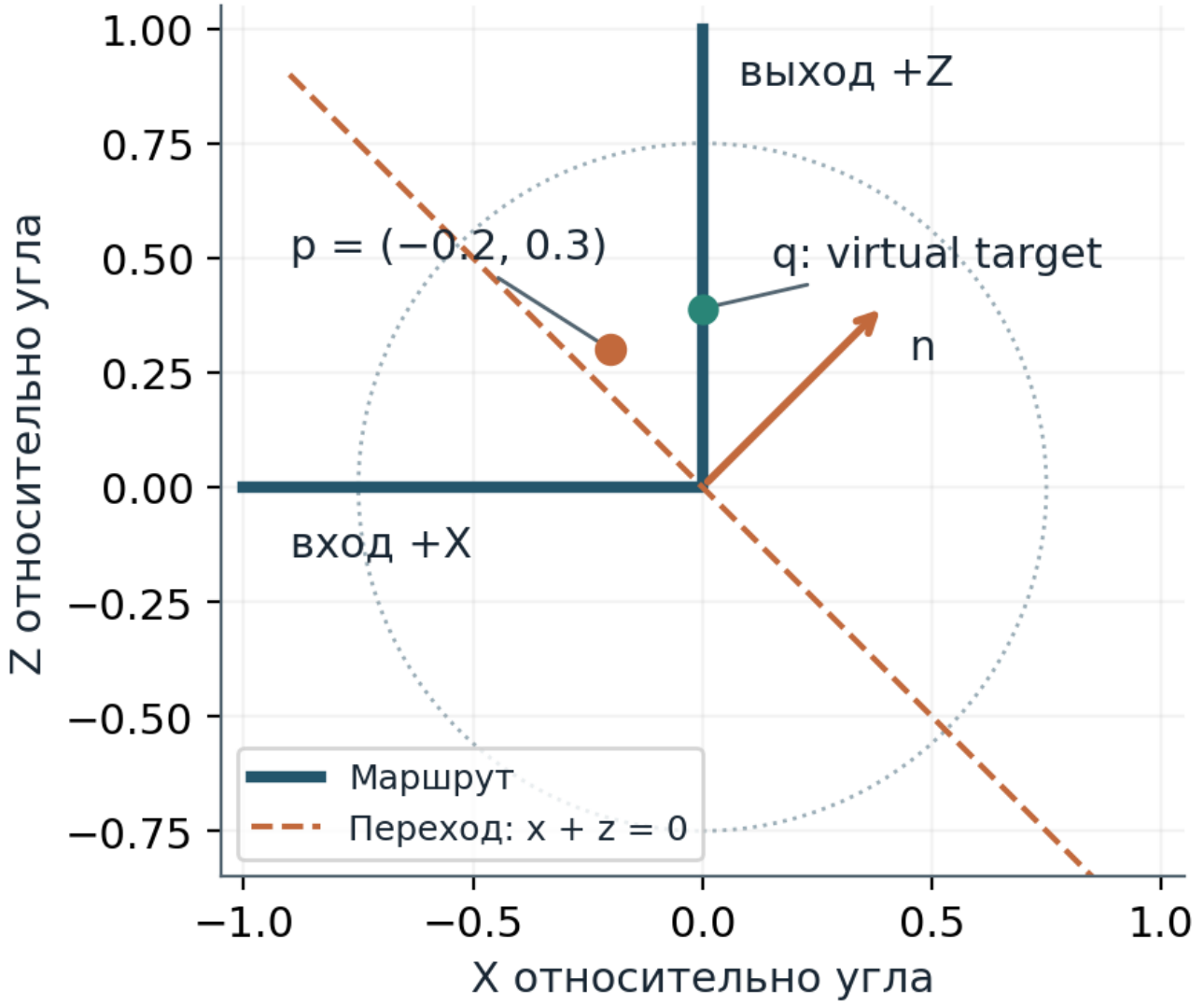


Рисунок 7. Плоскость перехода на повороте 90° и виртуальная цель.

При развороте почти на 180° сумма направлений вырождается. Код использует входящее направление как fallback-нормаль. Неконечные и слишком короткие сегменты не должны давать случайное нормализованное направление. Look-ahead-продвижение ограничено одним переходом за tick; обычное достижение нескольких совпавших или близких точек обрабатывается отдельным циклом.

Финиш завершается по arrival-контракту. Механизм углов не является общей гарантией успеха при любом overshoot последней точки. Если персонаж физически не может войти в acceptance radius, нужны диагностика маршрута, радиуса и контроллера, а не ожидание, что look-ahead завершит такой путь.

Код: GameplayRouteFollower.cppm, GameplaySteering.cppm, AIFollowRouteActionRuntime.cppm.

11. Steering: направление и величина команды

Базовые функции steering работают в горизонтальной плоскости XZ при вертикали +Y. Они возвращают `GameplayMovementIntent`: мировое направление, magnitude от 0 до 1 и флаг бега. Это управляющая команда для персонажа. Она не является физической силой и не задаёт гарантированную мгновенную скорость.

У Reynolds steering описывает локальное управление между выбором поведения и locomotion. Для чтения этой архитектурной границы полезна оригинальная работа Steering Behaviors for Autonomous Characters. Конкретные формулы ниже относятся к реализации CoreEngine и не претендуют на полный набор классических steering behaviors.

Seek и Flee

Для Seek вычисляется горизонтальная разность цели и позиции агента. Если расстояние вне acceptance radius с допуском 0.001, направление нормализуется, а magnitude равен 1. Внутри допуска результат — `Arrived` и нулевая команда. Seek не создаёт отдельной зоны плавного замедления.

$$\Delta = (x_t - x_p,\ 0,\ z_t - z_p), \qquad d = \|\Delta\|, \qquad \hat{v}_{seek} = \Delta / d$$

Flee использует противоположную разность — от угрозы к агенту — и тоже полную magnitude. При совпадении горизонтальных позиций направление неопределённо; текущий результат равен нулю. Здесь нет случайного направления выхода из совпадения. В примере агент в (0, 0, 0), цель в (3, 0, 4): d = 5, Seek выдаёт (0.6, 0, 0.8), Flee от той же точки — (−0.6, 0, −0.8).

Arrival

Arrival добавляет зону замедления между acceptance radius r и slowing radius R. Входные значения приводятся к r ≥ 0 и R ≥ r. Внутри r + 0.001 команда нулевая; при вырожденном интервале R−r или за пределами R — полная. В оставшейся области действует линейная шкала с нижней границей magnitude.

$$m(d) = \max\left(m_{min}, \operatorname{clamp}\left(\frac{d-r}{R-r}, 0, 1\right)\right)$$

Эта формула применяется только вне зоны прибытия и при ненулевом slowing interval. mmin предварительно ограничен диапазоном [0, 1]. По умолчанию r = 0.25, R = 1.0, mmin = 0.1. Нижняя граница помогает сохранить заметную команду до входа в acceptance; при самом прибытии magnitude всё равно становится нулевой.

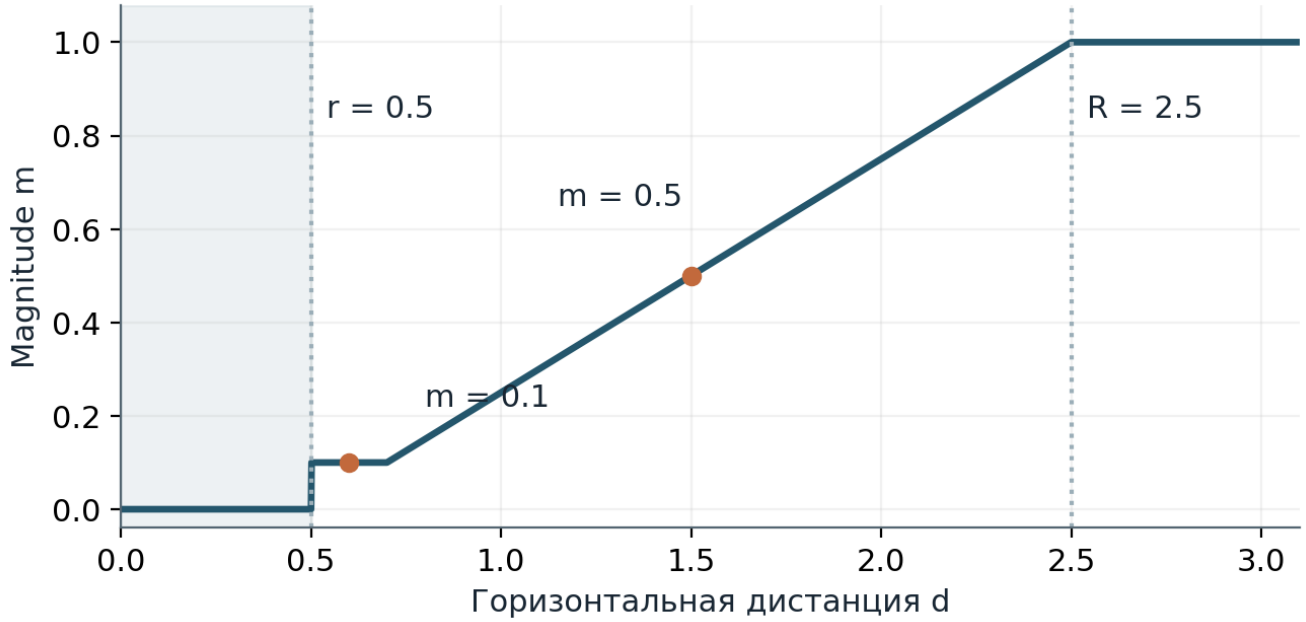


Рисунок 8. Учебный профиль Arrival при r = 0.5, R = 2.5 и minimum magnitude = 0.1.

На графике при d = 1.5 magnitude равна 0.5; при d = 0.6 линейная часть дала бы 0.05, поэтому применяется 0.1. Внутри acceptance — 0. Между остановкой и минимальным ненулевым значением есть небольшой скачок; это свойство формулы, а не непрерывный физический закон торможения. Фактическая скорость дополнительно зависит от motor и физического движения.

Применение к CharacterCommand

`ApplyGameplayMovementIntent` очищает осевые поля `moveInputX/Y`, проецирует направление на XZ, нормализует его и ограничивает magnitude. При отсутствии значимого направления или magnitude очищается движение и снимается `wantsRun`. Тем самым одна форма команды становится общей для Follow, Flee и движения по маршруту.

Есть важное различие с наблюдением `within_distance`: оно использует полное 3D-расстояние до сохранённой роли, а базовый steering — горизонтальную дистанцию. При существенном перепаде высоты follower может считать горизонтальное прибытие завершённым, а observation ещё не подтвердит факт. Высотные переходы и согласование контрактов прибытия требуют явного внимания автора поведения.

Код: GameplaySteering.cppm, GameplayGOAPSpatialComponents.cppm.

12. FollowTarget и FleeTarget во времени

FollowTarget читает текущие transforms агента и цели, вычисляет Seek и продолжает жить в состоянии `Running`, даже когда команда временно нулевая внутри acceptance radius. Такое действие означает «поддерживать следование», а не «однократно посетить точку». Если цель снова отдалится, очередной steering sample возобновит движение.

FleeTarget добавляет состояние `isFleeing` и два порога расстояния. В спокойном состоянии агент начинает убегать при d ≤ triggerRadius; во время бегства продолжает, пока d < safeRadius. Safe radius приводится как минимум к trigger radius. Между порогами результат зависит от предыдущего состояния.

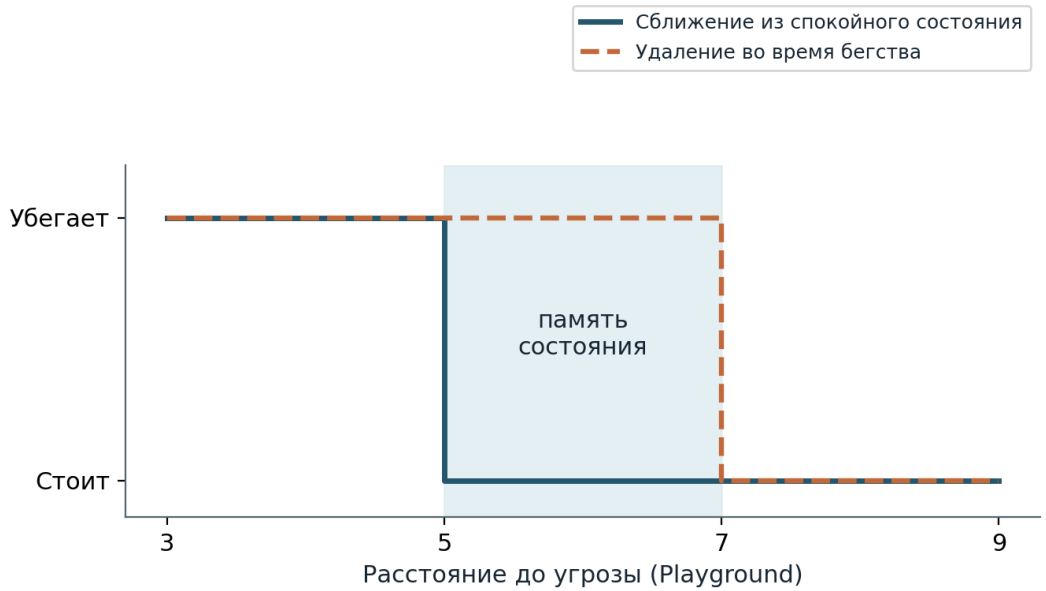


Рисунок 9. Гистерезис Flee: промежуточная зона сохраняет предыдущее состояние.

Настройки runtime по умолчанию — trigger 2, safe 3 и интервал пересчёта 0.05 секунды. В Steering Playground пороги заданы как 5 и 7. Поэтому при расстоянии 6 агент, который только наблюдает угрозу, ещё стоит, а уже убегающий продолжает. Один порог заставлял бы шум расстояния многократно включать и выключать бегство около границы.

Интервал steering управляет частотой чтения цели и пересчёта локального решения. На каждом tick runtime валидирует необходимые компоненты и применяет текущую сохранённую команду. При накопленном времени, достигшем интервала, sample обновляется; остаток времени сохраняется через `fmod`. Это не симуляция всех пропущенных подшагов после длинного кадра.

При ненулевой команде desired facing yaw вычисляется из итогового направления движения. Это особенно важно после avoidance: персонаж должен ориентироваться по скорректированному направлению, а не продолжать смотреть на геометрически прямую цель. Cancel очищает кеш движения и состояние avoidance; недействительная цель приводит к неуспеху и очистке доступной команды.

Эти runtimes дают практический механизм для подвижных целей. Если нужен GOAP-шаг с таким временем жизни, нужно определить его достижимый terminal-контракт и подходящий binding. Встроенный ассетный `move_to` не становится FollowTarget от одного лишь совпадения названия роли `target`.

Код: AIFollowTargetActionRuntime.cppm, AIFleeTargetActionRuntime.cppm, ai_steering_playground.scenario.json.

13. Obstacle avoidance: геометрия, память стороны и опора

Avoidance корректирует уже выбранное намерение движения. Он получает query-интерфейс к геометрии, параметры агента и небольшое состояние предыдущего выбора стороны. Поэтому сама функция не зависит от конкретного типа физического world. App предоставляет адаптер запросов к статической геометрии физической сцены.

Интеграционный helper берёт radius и полную высоту из физических параметров агента, строит origin на половине высоты над базой и использует фактическую горизонтальную скорость motor. Максимальный склон также берётся из физического профиля. Дублировать эти значения произвольными константами в behavior не требуется.

Три feelers и горизонт предсказания

При активном движении выполняются forward, left и right probes. Боковые направления повёрнуты на заданный угол относительно базового движения. По умолчанию угол равен 30°, боковая дальность — 2, базовая передняя — 3. Все три probe вычисляются до выбора ветви по переднему попаданию.

$$L_f = \max(L_0, v\tau), \qquad r_{query} = r_{character} + margin$$

Lf — дальность переднего запроса, L0 — минимальная дальность, v — фактическая горизонтальная скорость, τ — временной горизонт. При defaults τ = 0.75 с и L0 = 3: для v = 2 дальность остаётся 3, для v = 6 увеличивается до 4.5. Margin по умолчанию 0.02 добавляется к радиусу персонажа для clearance-запроса. Неконечные входы не должны порождать бесконечный probe.

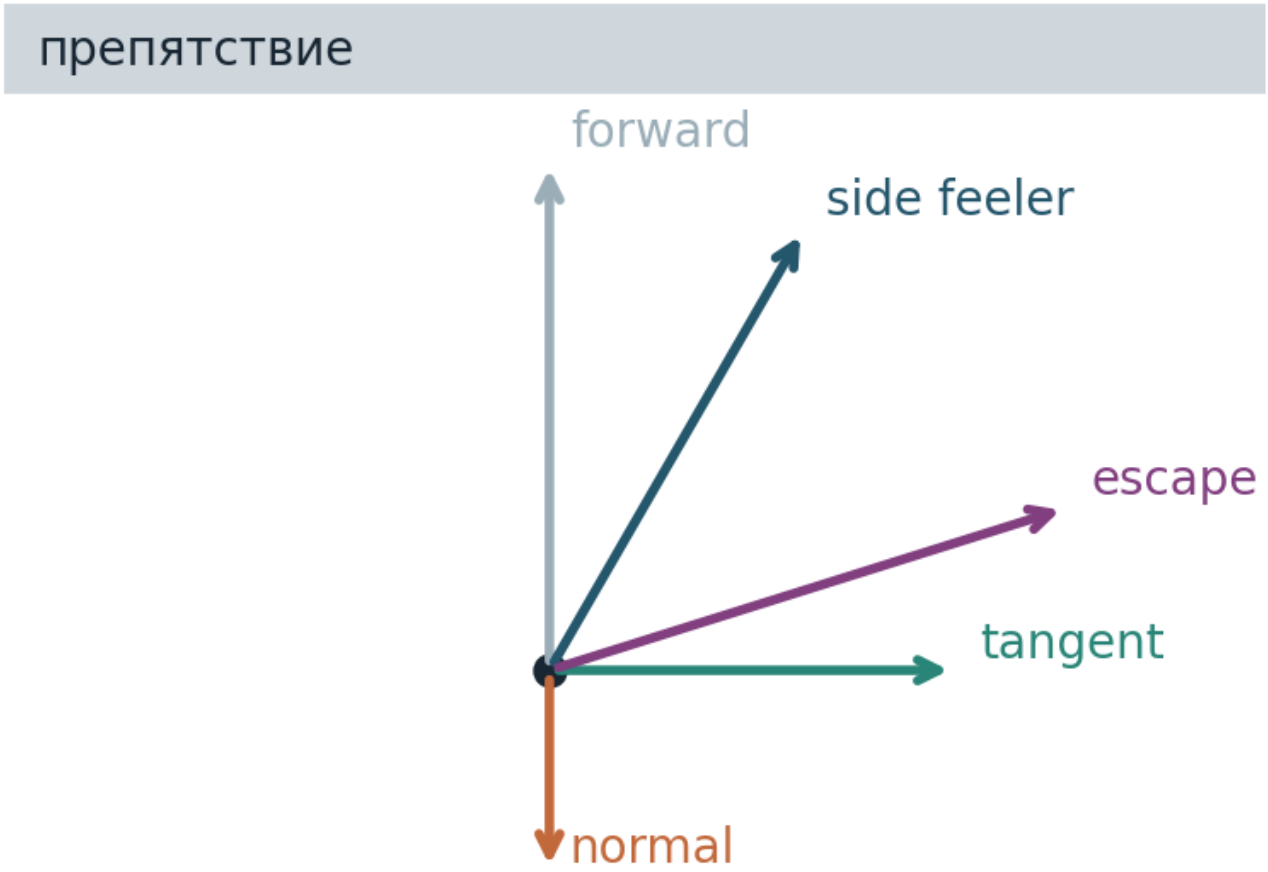


Рисунок 10. Боковой feeler, касательная поверхности и направление отделения формируют escape-кандидат.

Когда впереди препятствие, алгоритм строит левый и правый escape-кандидаты. Для выбранной стороны смешиваются направление feeler, ориентированная касательная поверхности и нормаль отделения от препятствия, затем результат нормализуется.

$$\hat{e} = \text{normalize}(0.60\hat{f}_{side} + 0.40\hat{t}_{side} + 0.30\hat{n}_{wall})$$

Вес 0.60 сохраняет желание двигаться вперёд и в выбранную сторону, 0.40 поддерживает скольжение вдоль стены, 0.30 отталкивает от её поверхности. Веса не являются вероятностями и не обязаны давать сумму 1: после смешивания используется единичное направление.

Касательная строится в XZ из нормали попадания, а её знак согласуется с выбранной стороной относительно forward. Смысл этого шага — устойчиво различать «вдоль стены вправо» и «вдоль той же стены влево». Без согласования ориентации одинаковая поверхность могла бы давать движение в противоположную сторону при небольшом изменении запроса.

Support проверяется по итоговому escape

Широкий боковой просвет сам по себе недостаточен: за ним может отсутствовать опора. Для каждого итогового escape-направления выполняется проверка поверхности под будущей позицией. Она делается после смешивания feeler и касательной, поэтому проверяет именно направление предполагаемого движения.

$$o_s = b + \hat{e} d_s + (0, h_s, 0), \qquad L_s = h_s + d_{drop}$$

b — база персонажа, ds — вынос вперёд, hs — подъём origin над базой, ddrop — допустимое падение опоры. При defaults ds = 0.75, hs = 0.25, ddrop = 0.35 длина нисходящего запроса равна 0.60. Это локальная проверка кандидата на ограниченном расстоянии, а не доказательство проходимости всего будущего маршрута.

Найденная опора принимается только при корректных конечных данных и допустимой дистанции. Если проверка walkability включена, нормаль нормализуется и сравнивается с максимальным склоном профиля:

$$\hat{n}_{support,\,y} \geq \cos(\theta_{max}) - \varepsilon$$

Для θmax = 45° порог примерно 0.7071 с числовым допуском. Нормаль (0, 1, 0) проходит; слишком наклонная или вырожденная нормаль — нет. Это проверка ориентации поверхности, а не визуального вида треугольника.



Рисунок 11. Допустимость опоры имеет приоритет над сравнением бокового просвета.

Выбор стороны и гистерезис

Если поддержан только один кандидат, выбирается он, даже когда прошлый выбор был другим. Если не поддержан ни один, команда обнуляется и запомненная сторона очищается. Если поддержаны оба, сравниваются clearance-оценки с преимуществом для уже выбранной стороны. Переключение требует строгого преимущества другой стороны больше `sideSwitchAdvantage`, по умолчанию 0.15. При первоначальном равенстве выбирается левая сторона.

Например, при левой сохранённой стороне оценки 1.00 и 1.10 оставляют левую; при 1.00 и 1.20 разрешается смена на правую. Это пример логики порога, а не единый масштаб для любого query. Когда впереди свободно, возвращается базовое движение и память стороны очищается. Память также сбрасывается при остановке и передаче управления traversal.

Границы текущей реализации

Support-ветвь включается при обходе обнаруженного переднего препятствия. Когда впереди свободно, базовая команда возвращается без такой проверки. Поэтому этот avoidance нельзя считать общим cliff guard, который останавливает любое движение к краю.

Метод `IGameplayObstacleQuery::ProbeSupport` имеет совместимое поведение по умолчанию с синтетической плоской опорой. Query, который не переопределяет его, не выполняет реальную геометрическую проверку земли. Для полноценной проверки нужен подключённый физический адаптер с собственной реализацией support.

Механизм локален. Он не гарантирует выход из тупика, глобальную перестройку маршрута, согласование встречных потоков или ORCA-подобное предотвращение столкновений толпы. Физический контроллер остаётся последней инстанцией допустимого перемещения.

Код: GameplayObstacleAvoidance.cppm, GameplayAgentObstacleAvoidance.cppm, GameplayPhysicsObstacleQuery.h.

14. Traversal, физика и фактически подключённые сервисы

Обычный сегмент маршрута исполняется через steering. Сегмент с traversal annotation переводит follower в `TraversalRequired` и отдаёт handle внешнему executor. Это может быть прыжок или проход через дверь. Ground steering на это время приостанавливается, состояние avoidance очищается. После успешного завершения executor follower подтверждает тот же link handle и переходит к следующему сегменту.

Link задаёт существующую возможность перехода в мире. В ассетной композиции traversal binding проверяет тип, handle и связанную целевую сущность. `.routes.json` аннотирует ребро; он не создаёт физическую дверь или jump link из одного текстового имени. Эти объекты должны быть подготовлены уровнем и lifecycle сценария.

`CompleteTraversal` означает, что внешняя система достигла конца аннотированного сегмента. Неуспех executor нельзя выдавать за обычное прибытие. При остановке сценария важно отменить активное действие и освободить связанное состояние до удаления объектов перехода.

Команда, движение и анимация

AI выбирает движение; motor и физический контроллер определяют реализованное перемещение. Возможны ненулевая команда и почти нулевая фактическая скорость у стены. В таком случае «план существует» и «steering выдаёт направление» ещё не означают физического прогресса. Признак `physicallyBlocked` служит диагностикой, но сам по себе не завершает Route runtime неуспехом и не запускает универсальное восстановление.

Анимация получает синхронизированное gameplay-состояние после физического обновления. Отсюда следует порядок диагностики: сравнить намерение, команду, фактическое движение и затем представление персонажа. Если визуальный бег происходит без перемещения, причина может находиться в коллизии или профиле, даже когда решение GOAP корректно.

Общий runtime не означает одинаковую композицию

В текущем `MoveToCapability::CreateBinding` передаются world, traversal links, executors, provider и reservations. Аргументы obstacle query, steering debug registry и navigation debug registry не передаются и остаются `nullptr`. `GameplayAIDecisionCreationContext` также не содержит этих сервисов.

Вход в движение	Obstacle query	Runtime debug движения	Planned-path inspection
Прямой запуск Follow/Flee/Route через GameplayRuntime	Передаётся	Передаются соответствующие registry	Не является GOAP-планом
`move_to` из текущей GOAP-композиции	Не передаётся	Не передаются steering/navigation registry	Доступен отдельно при `inspectPath`
Maze через прямой FollowRoute	Подключается общим прямым путём	Маршрут и steering доступны при включённых настройках	Не используется

Таким образом, общий `AIFollowRouteActionRuntime` умеет avoidance, но у нынешнего ассетного GOAP MoveTo эта возможность не подключена. Нельзя обещать, что GOAP-агент обходит препятствия теми же физическими probes, что и Maze или Steering Playground. Его физический контроллер продолжает обрабатывать коллизии, а сам маршрут может учитывать авторскую топологию; это другие механизмы.

Если проекту нужна одинаковая интеграция, следующим изменением должен стать явный контракт передачи query и debug-сервисов через creation context и binding. В этой редакции документации такое изменение не выполнялось и описано как дальнейшая работа.

Код: AIFollowRouteActionRuntime.cppm, JumpTraversalExecutor.cppm, DoorTraversalExecutor.cppm, GameplayGOAPMoveToComponent.cppm, AIMoveToActionBinding.cppm, GameplayAIDecisionCreationContext.cppm, GameplayRuntime_PublicApi.inl.

15. Авторинг поведения и development-сценарии

Самый короткий существующий пример новой композиции — Marker Visit. Его задача: бежать к доступному маркеру и завершить цель после подтверждённого прибытия. Он использует только зарегистрированные компоненты; отдельного доменного C++-класса для такого поведения не требуется.

Три файла Marker Visit

Следующие JSON соответствуют содержимому текущих файлов; изменены только пробелы и переносы. Пути отсчитываются от корня репозитория. Первая часть — `assets/ai/goap/marker_visit.goap.json`:

```
{
  "id": "marker_visit",
  "facts": [
    {"name": "markerReached", "type": "bool"},
    {"name": "markerPresent", "type": "bool"}
  ],
  "goals": [{
    "name": "visitMarker", "score": 1,
    "facts": [{"fact": "markerReached", "value": true}]
  }],
  "actions": [{
    "action": "move_to", "context": "visitMarker", "cost": 2,
    "preconditions": [{"fact": "markerPresent", "value": true}],
    "effects": [{"fact": "markerReached", "value": true}],
    "continuationConditions": [
      {"fact": "markerPresent", "value": true}
    ]
  }]
}
```

Definition не знает имени node уровня. Он связывает условие существования маркера, цель прибытия и ожидаемый эффект движения. Вторая часть — `assets/ai/behaviors/marker_visit.behavior.json`:

```
{
  "version": 1,
  "id": "marker_visit",
  "model": "goap",
  "definition": "ai/goap/marker_visit.goap.json",
  "observations": [
    {
      "type": "target_available", "target": "destination",
      "fact": "markerPresent", "requireInteractionPoint": true
    },
    {
      "type": "within_distance", "target": "destination",
      "fact": "markerReached", "radius": 0.8,
      "requireInteractionPoint": true
    }
  ],
  "capabilities": [{
    "type": "move_to", "context": "visitMarker",
    "source": "origin", "target": "destination",
    "acceptanceRadius": 0.8, "slowingRadius": 1.5,
    "wantsRun": true
  }]
}
```

Здесь каждый объявленный факт получил writer, а контекст `visitMarker` — capability. Радиусы движения и подтверждения согласованы для данного плоского примера. `requireInteractionPoint` превращает доступность в конкретное проверяемое состояние world. Последняя часть — `assets/ai/bindings/marker_visit.bindings.json`:

```
{
  "version": 1,
  "roles": [
    {"role": "origin", "node": "GOAP_Marker_Start"},
    {"role": "destination", "node": "GOAP_Marker_Goal"}
  ]
}
```

Bindings связывают роли с уровнем. Чтобы получить новый вариант поведения, нужно дать файлам и каталожной записи согласованные ID и ссылки, обеспечить nodes и необходимые компоненты world, а затем запускать ID решения через публичный API. Если меняется только размещение, меняются bindings и уровень; если меняется условие успеха, нужно пересмотреть факты, observations и эффекты вместе.

Последовательность добавления

1. Определить наблюдаемый критерий успеха и объявить только необходимые bool/int факты.

2. Описать действия, стартовые условия и ожидаемые эффекты; для длительных шагов отдельно выбрать continuation conditions.

3. Назначить каждому факту одного observer-writer и объявить зависимости readers.

4. Сопоставить каждому `(action, context)` зарегистрированную capability и реальные роли.

5. При необходимости добавить авторский route graph и bindings существующих traversal links.

6. Добавить запись каталога и development-сценарий с воспроизводимым lifecycle.

7. Проверить ошибки подготовки, фактическое достижение цели и поведение после Cancel/Reset.

Новый тип компонента проходит более длинный путь: owning payload, parser с проверкой полей, compiler, реализация интерфейса, регистрация и проверки контракта. Generic composition остаётся общей; привязка одного сценария не должна становиться веткой по его имени в `GameplayRuntime`.

Каталог и демонстрации

Решение или сценарий	Что проверяет
`marker_visit`	Минимальную самостоятельную композицию из ассетов и иной набор параметров
`target_recovery`	Continuation, временную потерю цели и восстановление retained instance
`access_key`	Маршрутные стоимости, сбор ресурса, покупку и доступ к цели
`access_key_reserved`	Конкуренцию за настроенные Interaction Points и освобождение reservation
`access_key_jump`	Сохранение traversal annotation при поиске и исполнении маршрута
Access Key Replanning	Изменение доступности ресурсов во время работы решения
Steering Playground	Follow/Flee, локальный avoidance, поворот и сохранение команды между samples
Navigation Pathfinding Maze	Построение пути по NavMesh и прямое исполнение FollowRoute

Первые пять строк соответствуют пяти решениям текущего `catalog.json`. Остальные — development-сценарии, а не дополнительные decision IDs. Это различие важно при запуске: ID сценария принадлежит App tooling, ID решения — factory registry AI.

Lifecycle сценария

Setup подготавливает world, сохраняет baseline и создаёт необходимые компоненты. Start отменяет прошлую работу, восстанавливает исходное состояние и запускает настоящий production API. Update выполняет сценарные вмешательства, например временное удаление доступности цели. Stop прекращает действие. Reset должен восстановить физическое, игровое и визуальное состояние, включая изменённые Interaction Points и reservations.

Физические параметры агента настраиваются до построения соответствующего NavMesh-профиля. Изменение radius после bake может сделать наблюдаемое движение несогласованным с путём. Для воспроизводимости сценарий хранит собственное состояние lifecycle в App; runtime Gameplay не должен получать поля наподобие «Maze уже запущен» или «пять секунд recovery истекли».

Код: marker_visit.goap.json, marker_visit.behavior.json, marker_visit.bindings.json, catalog.json, ai_goap_marker_visit.scenario.json, DevelopmentScenario.cpp, AppDevelopmentScenarioRuntime.cpp.

16. Наблюдаемость и диагностика

Отладка AI требует нескольких согласованных представлений. Факт цели объясняет выбор действия, planned path — намерение пространственного провайдера, runtime route — путь конкретного исполнителя, steering snapshot — локальную коррекцию, физическая скорость — реальный результат. Ни одно из них отдельно не доказывает корректность всей цепочки.

`AIDebugViewModel` и GOAP inspection дают доступ к фактам, выбранной цели и исполнению. Planned-path inspection включается через `inspectPath` в behavior и соответствующие capability interfaces. Для MoveTo это путь из авторского графа. Он не является новым NavMesh-запросом и не показывает в реальном времени все отклонения физического движения.

`GameplayNavigationDebugRegistry` хранит принадлежащую runtime копию активного маршрута. Публикация начинается при старте действия и очищается при завершении или отмене. `GameplaySteeringDebugRegistry` хранит базовую и итоговую команду, probes и сведения выбора avoidance. Эти данные появляются только при переданных и включённых registry.

Визуальные слои

`drawNavigationMesh` показывает заполненные цветные треугольники навигационной геометрии с небольшим вертикальным смещением. `drawNavigationPathDebug` показывает реальный маршрут, переданный исполнителю; текущая линия имеет пурпурный цвет. `drawAIPlannedPathDebug` включает представления AI-пути и steering telemetry в предусмотренном App-пути отрисовки.

В текущей реализации NavMesh overlay использует геометрию default-профиля, тогда как запрос конкретного агента может использовать профиль его физических размеров. Если нарисованный проход кажется свободным, а запрос возвращает отсутствие пути, нужно сравнить именно профили. Красивый overlay другого профиля не подтверждает проходимость для данного radius.

Доступность поля в debug registry не гарантирует, что отдельный UI-слой рисует его во всех режимах. Для GOAP MoveTo дополнительно действует граница подключения из раздела 14: planned-path inspection может работать при отсутствии steering/navigation debug publication.

Диагностическая матрица

Наблюдение	Что проверить сначала	Где искать
`StartAIDecision` вернул false	Режим игры, конфигурацию агента, каталог, diagnostic фабрики и первый plan result	Public API и asset composition
План есть, движения ещё нет	Ready-to-start после установки нового плана; запуск шага на следующем update	Decision runtime и execution bridge
Команда появляется рывками	Повторное применение cached intent после очистки команды кадра	Follow/Flee Tick и BeginFrame
Follower завершён, факт цели false	3D-радиус observation, высоту и сохранённую позицию роли	Spatial observation и Arrival
GOAP-агент упирается в стену	Авторский путь либо прямой fallback; отсутствующий query в composition	MoveTo provider и CreateBinding
Агент меняет сторону у препятствия	Support обоих кандидатов, clearance, commitment и порог 0.15	Obstacle avoidance snapshot
Баланс не изменился после события	Agent, subject, receiptId, признак уже подтверждённой операции	Resource ledger
Новая цель не восстановилась	Явный Cancel, удалённый instance либо новый entity handle вместо прежнего	Runtime lifecycle и role binding
В Maze нет пути через видимый проход	Физический radius, профиль запроса, bake и завершённость corridor	LevelNavigation и Recast adapter

Порядок расследования идёт от ближайшего достоверного факта. Сначала проверить, какая цель и какой шаг активны, затем найти контракт, который должен дать следующий результат. Если маршрут найден, полезно сравнить текущую точку, выбранный steering target и фактическую команду. При ненулевой команде и отсутствии перемещения следующий объект проверки — физика, а не стоимость GOAP-действия.

Код: AIDebugViewModel.cppm, GameplayGOAPInspection.cppm, GameplayGOAPPathInspection.cppm, GameplayNavigationDebug.cppm, GameplaySteeringDebug.cppm, LevelNavigation.cppm.

17. Проверки и известные ограничения

Документ подготовлен по закреплённому commit ветки Deferred. При подготовке выполнен `tests/tools/run_goap_asset_portable_checks.py`: результат — **623 успешные проверки authoring, templates и dependency ordering**. Harness использует производственные parser и код подготовки, снимая module declarations для переносимой сборки GCC; callbacks создания runtime-компонентов в нём инертны и не вызываются.

Проверка запущена с AddressSanitizer и UndefinedBehaviorSanitizer; leak detection отключён через `ASAN_OPTIONS=detect_leaks=0`, поскольку LeakSanitizer не работал в окружении выполнения. Это не полный запуск GoogleTest/MSVC, не проверка физического движения и не визуальное прохождение Maze. Полная сборка движка и development-сцены при подготовке документа не запускались.

Где уже описаны проверяемые контракты

Файл тестов	Область
`TestGameplayGOAPDecisionTemplate.cpp`	Типизированные расширения, integer inputs, смешанные циклы зависимостей, изоляция instance и cost overrides
`TestGameplayAIAssetComposition.cpp`	Recovery, время жизни providers, ошибки ролей, единственный writer, транзакционная регистрация
`TestGameplayGOAPResourceComponents.cpp`	Баланс, переполнение, receipts, зависимости в одном update, reservations и traversal graph
`TestAIDecisionRuntime.cpp`	Выбор цели, неуспешная замена, перепланирование и различие стартовых условий/исполнения

Наличие этих тестов установлено чтением исходников. Их полный runtime-набор здесь не выполнялся. Переносимый harness является отдельной проверкой с ограниченным охватом.

Воспроизводимая ручная проверка

Для Marker Visit полезно подтвердить один старт, одно достижение маркера и отсутствие движения после Stop. Для Target Recovery — движение до временной потери цели, отмену текущего шага, отсутствие движения во время недоступности и новый старт после её восстановления. Отдельно стоит проверить начальный Start при недоступной цели: ожидается отказ без retained instance.

Для Access Key требуется наблюдать сбор двух различных credits, списание цены и единичное подтверждение receipt. Повтор того же события не должен выдавать дополнительный ресурс или повторную покупку. Для варианта reservations нужна конкуренция агентов за одну настроенную цель и освобождение владения при Cancel.

Maze проверяет путь от старта до финиша, прохождение углов, остановку внутри конечного acceptance и очистку debug route после Stop/Reset. Steering Playground позволяет отдельно сравнить Follow, оба порога Flee и смену стороны у препятствия с допустимой и недопустимой опорой. Эти процедуры описывают ожидаемую проверку в приложении, а не результаты выполненного визуального теста.

Ограничения, которые влияют на дизайн

Текущее ограничение	Последствие для проекта
Нет бюджета GOAP-поиска и backoff неуспешных попыток	Размер домена и частоту перепланирования нужно контролировать
Позиции ролей и entity handles разрешаются при создании instance	Подвижные и пересоздаваемые цели требуют другого контракта
GOAP MoveTo не получает obstacle query/debug services	Возможности общего Route runtime подключены не одинаково во всех входах
Условия продолжения проверяются на ветви неизменившейся цели	Сохранение старого исполнения при смене цели имеет описанную особенность
Support-проверка относится к escape при препятствии	Она не обеспечивает общий запрет шага с края
`physicallyBlocked` — диагностика	Нет универсального автоматического восстановления из физического застревания
Observation использует 3D, steering — XZ	Контракт прибытия нужно согласовать для перепадов высоты
Snapshot template без автоматического hot reload	Изменения ассетов требуют явного пересоздания registry/решения

Наиболее прямые направления дальнейшей работы следуют из этих границ: передача query и registry в composition, явное различие статической роли и отслеживаемой цели, наблюдаемая политика stuck recovery и ограниченный по бюджету поиск. Каждое изменение должно иметь собственный runtime-контракт и сценарий проверки. Это предложения для следующей редакции, а не список уже подключённых возможностей.

Код: run_goap_asset_portable_checks.py, TestGameplayGOAPDecisionTemplate.cpp, TestGameplayAIAssetComposition.cpp, TestGameplayGOAPResourceComponents.cpp, TestAIDecisionRuntime.cpp, README.md.

18. Точки входа, термины и источники

Публичные точки входа

API	Назначение
`GameplayRuntime::StartAIDecision`	Создать решение по каталожному ID и выполнить синхронную стартовую проверку
`GameplayRuntime::CancelAIDecision`	Для entity: отменить и удалить принадлежащий runtime instance
`GameplayRuntime::GetAIDecisionStatus`	Получить статус решения агента
`GameplayRuntime::GetAIDecisionObservedState`	Получить доступ к наблюдаемым фактам существующего решения
`GameplayRuntime::StartAIFollowRoute`	Запустить готовый маршрут напрямую
`GameplayRuntime::StartAIMoveTo`	Запустить MoveTo через прямой путь интеграции
`GameplayRuntime::StartAIFollowTarget`	Поддерживать следование живой цели
`GameplayRuntime::StartAIFleeTarget`	Поддерживать бегство с двумя порогами
`GameplayRuntime::CancelAIAction`	Прекратить текущую конкретную AI-задачу агента
`GameplayRuntime::GetAIActionStatus`	Проверить статус конкретного действия, отдельно от GOAP-плана

Таблица описывает назначение, а не копирует сигнатуры overloads. Точные параметры находятся в GameplayRuntime_PublicApi.inl и объявлении GameplayRuntime.cppm. Отмена action и отмена decision имеют разный масштаб; оставленное активным решение способно инициировать следующую работу по своему lifecycle.

Словарь

Термин	Значение в этом документе
Fact	Типизированное bool/int значение модели агента
Goal	Желаемые булевы факты и правила приоритета
Semantic action	Зарегистрированный вид возможности, например `move_to` или `purchase`
Context	Конкретизация семантического действия внутри определения
Capability / provider	Компонент, который связывает контексты с исполнимыми запросами и bindings
Binding	Переход от семантического шага плана к конкретному action runtime
Observation	Единственный производитель своих фактов из world/events
Reaction	Обработка подтверждённых событий с дополнительными игровыми последствиями
Template	Подготовленный неизменяемый снимок конфигурации без живого агента
Instance	Данные, наблюдения и исполнение одного решения одного агента
Route	Точки и аннотации пространственного движения
Steering	Локальная команда направления и величины движения
Traversal	Исполнение специального перехода по link через внешний executor
Reservation	Временное владение Interaction Point в ходе исполнения

Основание редакции

Главный источник — CoreEngineModule, commit 55de4ab55c48. Файлы в списке ниже ведут к этому commit, поэтому ссылки не меняют смысл при дальнейшем продвижении Deferred. При следующем обновлении документа следует заново проверить composition, порядок кадра и фактическую передачу сервисов в bindings.

При структуре и проверке документа учтены предоставленные правила `COREENGINE_DOCUMENTATION_RULES (1).md` и `CoreEngine_Development_Test_Scenario_Rules.txt`. Историческая дорожная карта `goap_development_roadmap.md` использована только как контекст; она не определяет статус текущего кода. Прежний рассказ GOAP/Steering не является спецификацией после миграции на новую композицию.

Внешние первоисточники нужны для теоретических терминов: STRIPS, Fikes и Nilsson, Steering Behaviors, Reynolds, Detour NavMesh Query. Поведение CoreEngine, конкретные значения и ограничения подтверждаются его исходниками.

Карта исходников

1. src/Gameplay/AI/Composition/GameplayAIBuiltinDecisions.cppm

2. src/Gameplay/AI/Composition/GameplayGOAPAssetComposition.cppm

3. src/Gameplay/AI/Runtime/GameplayAIDecisionCreationContext.cppm

4. src/Gameplay/AI/GOAP/AIAgentWorldState.cppm

5. src/Gameplay/AI/GOAP/Authoring/GameplayGOAPDefinitionCompiler.cppm

6. assets/ai/goap/access_key.goap.json

7. src/Gameplay/AI/GOAP/AIDecisionContracts.cppm

8. src/Gameplay/AI/GOAP/AIStateOperations.cppm

9. src/Gameplay/AI/GOAP/AIGoalSelection.cppm

10. assets/ai/goap/target_recovery.goap.json

11. src/Gameplay/AI/GOAP/AIPlanner.cppm

12. src/Gameplay/AI/Composition/GameplayGOAPDecisionTemplate.cppm

13. src/Gameplay/AI/Composition/GameplayGOAPCompositionRegistry.cppm

14. src/Gameplay/AI/GOAP/Authoring/GameplayAIDecisionAsset.cppm

15. src/Gameplay/AI/Composition/GameplayGOAPObservationOrder.cppm

16. src/Gameplay/AI/Composition/Components/GameplayGOAPSpatialComponents.cppm

17. src/Gameplay/AI/Composition/Components/GameplayGOAPResourceComponents.cppm

18. src/Gameplay/AI/GOAP/GameplayGOAPDecisionInstance.cppm

19. src/Gameplay/Events/GameplayWorldEvent.cppm

20. src/Gameplay/AI/GOAP/AIDecisionRuntime.cppm

21. src/Gameplay/Runtime/GameplayRuntime_PublicApi.inl

22. src/Gameplay/Runtime/GameplayRuntime_FrameLifecycle.inl

23. assets/ai/behaviors/target_recovery.behavior.json

24. assets/development/
ai_goap_target_recovery.scenario.json

25. src/Gameplay/AI/GOAP/
AIPlanExecution.cppm

26. src/Gameplay/AI/GOAP/
AIPlanExecutionBridge.cppm

27. src/Gameplay/AI/Runtime/
AISystem.cppm

28. src/App/AppLifecycleImpl/
AppLifecycle_TickImpl.inl

29. src/Gameplay/AI/Composition/Components/
GameplayGOAPMoveToComponent.cppm

30. src/Gameplay/AI/Navigation/
AIMoveToActionBinding.cppm

31. src/Gameplay/Navigation/
GameplayRoute.cppm

32. src/Gameplay/Navigation/
GameplayRouteSearch.cppm

33. src/App/
LevelNavigation.cppm

34. src/Navigation/Recast/
Navigation.cpp

35. assets/development/
navigation_pathfinding_maze.scenario.json

36. assets/levels/
navigation_pathfinding_maze.level.json

37. src/Gameplay/Navigation/
GameplayRouteFollower.cppm

38. src/Gameplay/Navigation/
GameplaySteering.cppm

39. src/Gameplay/AI/Navigation/
AIFollowRouteActionRuntime.cppm

40. src/Gameplay/AI/Navigation/
AIFollowTargetActionRuntime.cppm

41. src/Gameplay/AI/Navigation/
AIFleeTargetActionRuntime.cppm

42. assets/development/
ai_steering_playground.scenario.json

43. src/Gameplay/Navigation/
GameplayObstacleAvoidance.cppm

44. src/Gameplay/AI/Navigation/
GameplayAgentObstacleAvoidance.cppm

45. src/App/
GameplayPhysicsObstacleQuery.h

46. src/Gameplay/Traversal/
JumpTraversalExecutor.cppm

47. src/Gameplay/Traversal/
DoorTraversalExecutor.cppm

48. assets/ai/goap/
marker_visit.goap.json

49. assets/ai/behaviors/
marker_visit.behavior.json

50. assets/ai/bindings/
marker_visit.bindings.json

51. assets/ai/decisions/
catalog.json

52. assets/development/
ai_goap_marker_visit.scenario.json

53. src/App/Development/
DevelopmentScenario.cpp

54. src/App/Development/
AppDevelopmentScenarioRuntime.cpp

55. src/Gameplay/AI/GOAP/Debug/
AIDebugViewModel.cppm

56. src/Gameplay/AI/GOAP/Debug/
GameplayGOAPInspection.cppm

57. src/Gameplay/AI/GOAP/Debug/
GameplayGOAPPathInspection.cppm

58. src/Gameplay/Navigation/
GameplayNavigationDebug.cppm

59. src/Gameplay/Navigation/
GameplaySteeringDebug.cppm

60. tests/tools/
run_goap_asset_portable_checks.py

61. tests/unit/GameplayTests/AI/
TestGameplayGOAPDecisionTemplate.cpp

62. tests/unit/GameplayTests/AI/
TestGameplayAIAssetComposition.cpp

63. tests/unit/GameplayTests/AI/
TestGameplayGOAPResourceComponents.cpp

64. tests/unit/GameplayTests/AI/
TestAIDecisionRuntime.cpp

65. tests/
README.md

66. src/Gameplay/Runtime/
GameplayRuntime.cppm